

Actividad Evaluable 2

2026-01-12

Instalación y carga de las librerías para ser usadas a lo largo de la ejecución del código.

```
#install.packages("rvest")
#install.packages("httr")
#install.packages("dplyr")
#install.packages("ggplot2")
#install.packages("forcats")
#install.packages("patchwork")
```

```
library("rvest")
library("httr")
library("dplyr")
```

```
##
## S'està adjuntant el paquet: 'dplyr'
```

```
## Els següents objectes estan emmascarats des de 'package:stats':
##
##      filter, lag
```

```
## Els següents objectes estan emmascarats des de 'package:base':
##
##      intersect, setdiff, setequal, union
```

```
library("stringr")
library("ggplot2")
library("forcats")
library("patchwork")
```

Pregunta 1:

Queremos programar un programa de tipo *web scraping* con el que podamos obtener una página web, mediante su URL, y poder analizar su contenido HTML con tal de extraer datos e información específica. Nuestro programa ha de ser capaz de cumplir con los siguientes pasos:

1. Descargar la página web de la URL indicada, y almacenarlo en un formato de R apto para ser tratado. El primer paso para realizar tareas de *crawling* y *scraping* es poder descargar los datos de la web. Para esto usaremos la capacidad de R y de sus librerías (httr y XML) para descargar webs y almacenarlas en variables que podamos convertir en un formato fácil de analizar (p.e. de HTML a XML).

Primero de todo guardamos la URL a analizar dentro de una variable para poder usarla más adelante.

```
url <- "https://www.mediawiki.org/wiki/MediaWiki"
```

Como nosotros usamos la librería **rvest**, podemos obtener los datos de la página haciendo un simple *read_html*.

```
html <- read_html(url)
#html

html |> html_element("h1")
```

```
## {html_node}
## <h1 id="firstHeading" class="firstHeading mw-first-heading">
```

2. Analizar el contenido de la web, buscando el título de la página (que en HTML se etiqueta como “title”). En las cabeceras web encontramos información como el título, los ficheros de estilo visual, y meta-información como el nombre del autor de la página, una descripción de esta, el tipo de codificación de esta, o palabras clave que indican qué tipo de información contiene la página. Una vez descargada la página, y convertida a un formato analizable (como XML), buscaremos los elementos de tipo “title”. P.e. <title>Titulo de Página</title>.

Para obtener el título de la página web, buscamos con *html_elements* donde aparece *title* y obtenemos su texto, que guardamos en una variable por si nos hace falta más adelante.

```
title <- html |> html_elements("title") |> html_text2()
title
```

```
## [1] "MediaWiki"
```

3. Analizar el contenido de la web, buscando todos los enlaces (que en HTML se etiquetan como “a”), buscando el texto del enlace, así como la URL. Vamos a extraer, usando las funciones de búsqueda XML, todos los enlaces que salen de esta página con tal de listarlos y poder descargarlas más tarde. Sabemos que estos son elementos de tipo “a”, que tienen el atributo “href” para indicar la URL del enlace. P.e. Texto del Enlace. Del enlace nos quedaremos con la URL de destino y con el valor del enlace (texto del enlace).

Para la realización de este ejercicio, donde obtenemos las URL y su texto correspondiente, lo que hacemos es: primero obtenemos las URL con *html_attr*, luego obtenemos de la misma forma el texto (lo hacemos por separado ya que a partir del *href* no podemos obtener el texto) pero en este caso obtenemos solo el texto correspondiente. Finalmente creamos un *data frame* que contiene ambas columnas.

```
urls <- html |> html_elements("a") |> html_attr("href")
texto <- html |> html_elements("a") |> html_text2()

datos <- data.frame(
  URL = urls,
  Texto = texto,
  stringsAsFactors = FALSE
)
head(datos, 10)
```

##	URL	Texto
## 1	#bodyContent	Jump to content
## 2	/wiki/MediaWiki	Main page
## 3	/wiki/Download	Get MediaWiki
## 4	/wiki/Special:MyLanguage/Category:Extensions	Get extensions
## 5	https://techblog.wikimedia.org/	Tech blog
## 6	/wiki/Special:MyLanguage/How_to_contribute	Contribute
## 7	/wiki/Special:MyLanguage/Help:Contents	User help
## 8	/wiki/Special:MyLanguage/Manual:FAQ	FAQ
## 9	/wiki/Special:MyLanguage/Manual:Contents	Technical manual
## 10	/wiki/Project:Support_desk	Support desk

4. Generar una tabla con cada enlace encontrado, indicando el texto que acompaña el enlace, y el número de veces que aparece un enlace con ese mismo objetivo. En este paso nos interesa reunir los datos obtenidos en el anterior paso. Tendremos que comprobar, para cada enlace, cuántas veces aparece.

Ahora para generar la tabla con las repeticiones, usamos la función `table` de R. Después de realizar el conteo de repeticiones, lo transformamos en *data frame*, de manera que nos queda la tabla que teníamos anteriormente, con una columna adicional que es **Repeticiones**. Para eliminar todos aquellos datos que tienen 0 repeticiones, usamos la función `subset` que solamente deja las filas que cumplen con la condición especificada.

```
tabla_freq <- table(datos)
datos <- as.data.frame(tabla_freq, responseName = "Repeticiones")

datos <- subset(datos, Repeticiones > 0)

head(datos, 10)
```

##	URL	Texto	Repeticiones
## 1	#		7
## 20	/wiki/File:Wikimedia_Hackathon_2025_Group_photo_from_drone.jpg		1
## 21	/wiki/Help:Contents		1
## 22	/wiki/Help:Editing_pages		1
## 23	/wiki/Help:Navigation		1
## 24	/wiki/How_to_report_a_bug		1
## 25	/wiki/MediaWiki		1
## 26	/wiki/Professional_development_and_consulting		1
## 30	/wiki/Project:Language_policy		1
## 69	/wiki/Wikimedia_Hackathon_2025		1

5. Para cada enlace, seguirlo e indicar si está activo (podemos usar el código de status HTTP al hacer una petición a esa URL). En este paso podemos usar la función `HEAD` de la librería “`httr`”, que en vez de descargarse la página como haría `GET`, solo consultamos los atributos de la página o fichero destino. `HEAD` nos retorna una lista de atributos, y de entre estos hay uno llamado “`header`” que contiene más atributos sobre la página buscada. Si seguimos podemos encontrar el “`status_code`” en “`resultado$status_code`”. El “`status_code`” nos indica el resultado de la petición de página o fichero. Este código puede indicar que la petición ha sido correcta (200), que no se ha encontrado (404), que el acceso está restringido (403), etc.
 - Tened en cuenta que hay enlaces con la URL relativa, con forma “`/xxxxxx/xxxxx/a.html`”. En este caso, podemos indicarle como “`handle`” el dominio de la página que estamos tratando, o añadirle el dominio a la URL con la función “`paste`”.
 - Tened en cuenta que puede haber enlaces externos con la URL absoluta, con forma “`http://xxxxxx/xxxx/a.html`” (o `https`), que los trataremos directamente.
 - Tened en cuenta que puede haber enlaces que apunten a subdominios distintos, con forma “`//subdominio/xxxx/xxxx/a.html`”. En este caso podemos adjuntarle el prefijo “`https:`” delante, convirtiendo la URL en absoluta.
 - Tened en cuenta URLs internas con tags, como por ejemplo “`#search-p`”. Estos apuntan a la misma página en la que estamos, pero diferente altura de página. Equivale a acceder a la URL relativa de la misma página en la que estamos. Es recomendado poner un tiempo de espera entre petición y petición de pocos segundos (comando “`Sys.sleep`”), para evitar ser “baneados” por el servidor. Para poder examinar las URLs podemos usar expresiones regulares, funciones como “`grep`”, o mirar si en los primeros caracteres de la URL encontramos “`//`” o “`http`”. Para tratar las URLs podemos usar la ayuda de la función “`paste`”, para manipular cadenas de caracteres y poder añadir prefijos a las URLs si fuera necesario.

Primero hacemos una copia del *data frame* para poder modificar hacer modificaciones de las URL y que no tenga impacto para los siguientes ejercicios.

```
copia_datos <- datos
```

Luego sustituimos los `#` por la URL que habíamos guardado al principio, ya que los `#` significan que se está apuntando a esa misma página, pero a alturas distintas de esta.

```
datos$URL <- gsub("#", url, datos$URL)
head(datos)
```

	URL	Texto
## 1	https://www.mediawiki.org/wiki/MediaWiki	
## 20	/wiki/File:Wikimedia_Hackathon_2025_Group_photo_from_drone.jpg	
## 21	/wiki/Help:Contents	
## 22	/wiki/Help:Editing_pages	
## 23	/wiki/Help:Navigation	
## 24	/wiki/How_to_report_a_bug	
##	Repeticiones	
## 1	7	
## 20	1	
## 21	1	
## 22	1	
## 23	1	
## 24	1	

Luego, para tratar las URL relativas y las absolutas que empiecen por `//`, realizamos una búsqueda con sustitución para los valores del *data frame* que cumplan las siguientes condiciones:

1. Que empiecen por `//`.
2. Que empiecen por `/`.

La razón del orden es básicamente porque si se hiciera del revés afectaría a las URL que tuviesen los `//`.

```
base_url <- "https:"
main_url <- "https://www.mediawiki.org"

datos <- datos %>%
  mutate(URL = if_else(
    condition = str_starts(URL, "//"),
    true       = str_c(base_url, URL),
    false      = URL
  ))

datos <- datos %>%
  mutate(URL = if_else(
    condition = str_starts(URL, "/"),
    true       = str_c(main_url, URL),
    false      = URL
  ))
```

Seguidamente vamos iterando por todas las filas del *data frame* y realizamos una conexión a la URL (usando la función *session* de **rvest** y guardamos el atributo `status_code` de la respuesta, luego se realiza una pequeña pausa para no estresar el servidor.

Finalmente volvemos a cambiar los datos que recién acabamos de tratar por los originales y guardamos los datos en una nueva columna que nombramos **Estado**.

```
s <- c()
for (i in 1:length(datos$URL)) {
  s[i] <- as.character(session(as.character(datos$URL[i]))$response$status_code)
  Sys.sleep(0.1)
}
```

```
## Warning in session_set_response(x, resp): Not Found (HTTP 404).
## Warning in session_set_response(x, resp): Not Found (HTTP 404).
```

```
datos_limpios <- datos
datos <- copia_datos

datos$Estado <- s

head(datos)
```

```
##                               URL Texto
## 1                               #
## 20 /wiki/File:Wikimedia_Hackathon_2025_Group_photo_from_drone.jpg
## 21                               /wiki/Help:Contents
## 22                               /wiki/Help:Editing_pages
```

```
## 23                               /wiki/Help:Navigation
## 24                               /wiki/How_to_report_a_bug
##   Repeticiones Estado
## 1           7    200
## 20          1    200
## 21          1    200
## 22          1    200
## 23          1    200
## 24          1    200
```

Pregunta 2:

Elaborad, usando las librerías de gráficos base y qplot (ggplot2), una infografía sobre los datos obtenidos. Tal infografía será una reunión de gráficos donde se muestren los siguientes detalles:

1. Un histograma con la frecuencia de aparición de los enlaces, pero separado por URLs absolutas (con “http...”) y URLs relativas.

Para conseguir las URL relativas y absolutas realizamos el siguiente cálculo:

1. **Absolutas** = URL que **no** empiezan por / + URL que **no** empiezan por #.
2. **Relativas** = URL que empiezan por / o URL que empiezan por #.

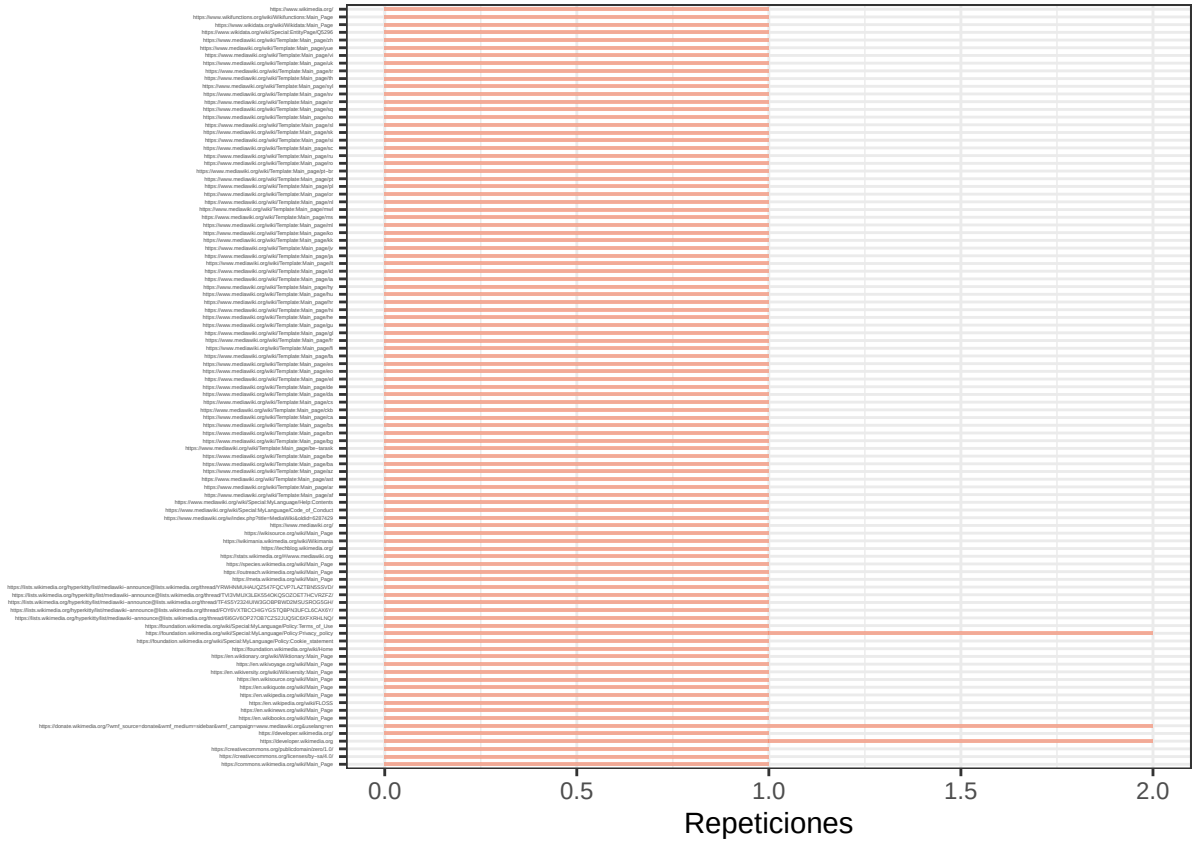
```
absolutas <- datos[!str_starts(datos$URL, "/") & !str_starts(datos$URL, "#"), ]
relativas <- datos[str_starts(datos$URL, "/") | str_starts(datos$URL, "#"), ]
```

Entonces una vez hecha la división creamos las dos tablas donde el eje X serán nuestras URL y el eje Y serán las veces que aparecen o *Repeticiones*.

```
barplot_absolutas <- absolutas %>%
  ggplot(aes(x=URL, y=Repeticiones)) +
  geom_bar(stat="identity", fill="#f68060", alpha=.6, width=.4) +
  coord_flip() +
  theme_bw() +
  # Reduce el tamaño del texto del eje Y (que ahora está a la izquierda)
  theme(axis.text.y = element_text(size = 2))
```

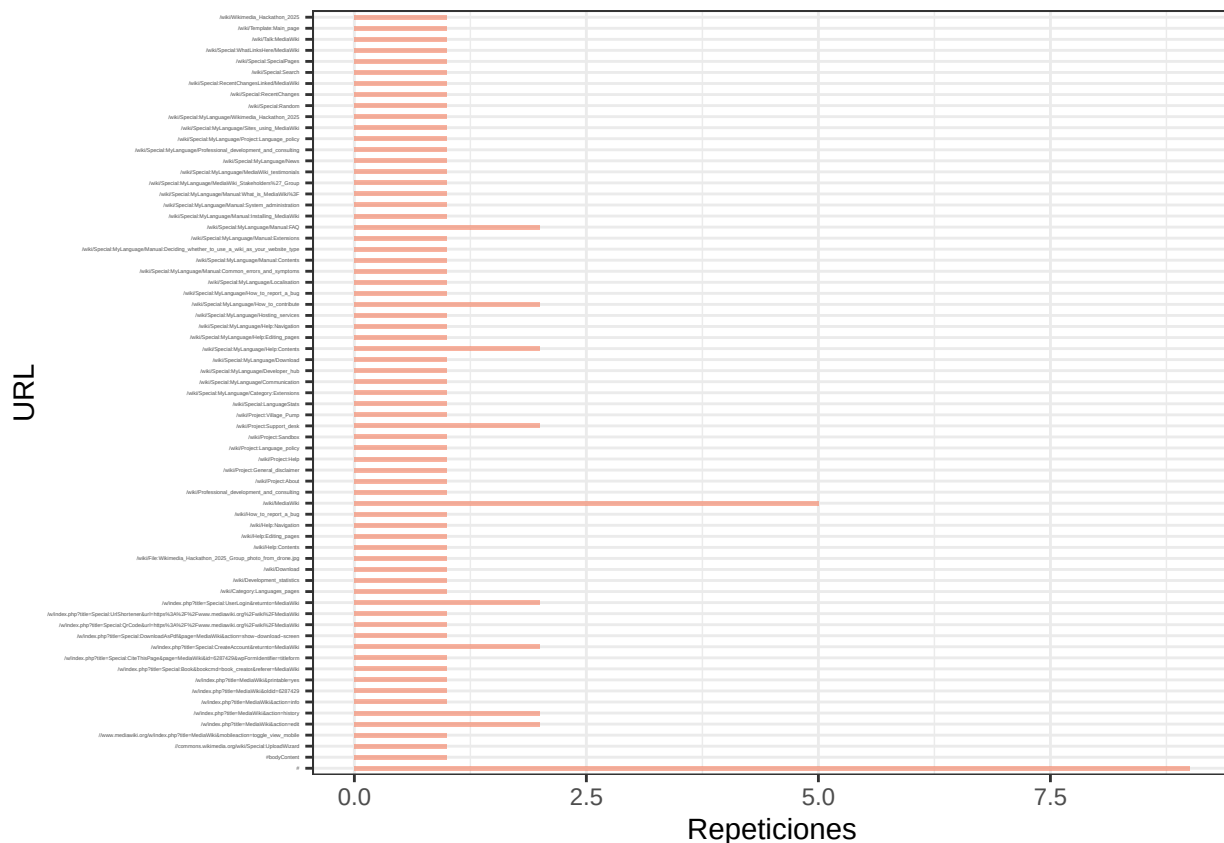
```
barplot_absolutas
```

URL



```
barplot_relativas <-relativas %>%
  ggplot(aes(x=URL, y=Repeticiones)) +
  geom_bar(stat="identity", fill="#f68060", alpha=.6, width=.4) +
  coord_flip() +
  theme_bw() +
  # Reduce el tamaño del texto del eje Y (que ahora está a la izquierda)
  theme(axis.text.y = element_text(size = 2))

barplot_relativas
```



2. Un gráfico de barras indicando la suma de enlaces que apuntan a otros dominios o servicios (distinto a <https://www.mediawiki.org> en el caso de ejemplo) vs. la suma de los otros enlaces. Aquí queremos distinguir enlaces que apuntan a mediawiki versus el resto. Sabemos que las URLs relativas ya apuntan dentro, por lo tanto hay que analizar las URLs absolutas y comprobar que apunten a <https://www.mediawiki.org>.

Para el siguiente gráfico de barras, decidimos almacenar las **Repeticiones** de las URL que apuntan a otros dominios de las que apuntan al mismo. Para ello realizamos los siguientes cálculos:

1. **Dominio:** Es la suma de las Repeticiones de URLs **Relativas** + suma de las Repeticiones de URLs **Absolutas** que empiezan por <https://www.mediawiki.org>.
2. **Otros:** Es la suma de las Repeticiones de URLs **Absolutas** que **NO** empiezan por <https://www.mediawiki.org>.

```
dominio <- sum(relativas$Repeticiones) + sum(absolutas[str_starts(absolutas$URL, main_url), "Repeticiones"])
otros <- sum(absolutas[!str_starts(absolutas$URL, main_url), "Repeticiones"])
```

```
total <- sum(datos$Repeticiones)
total
```

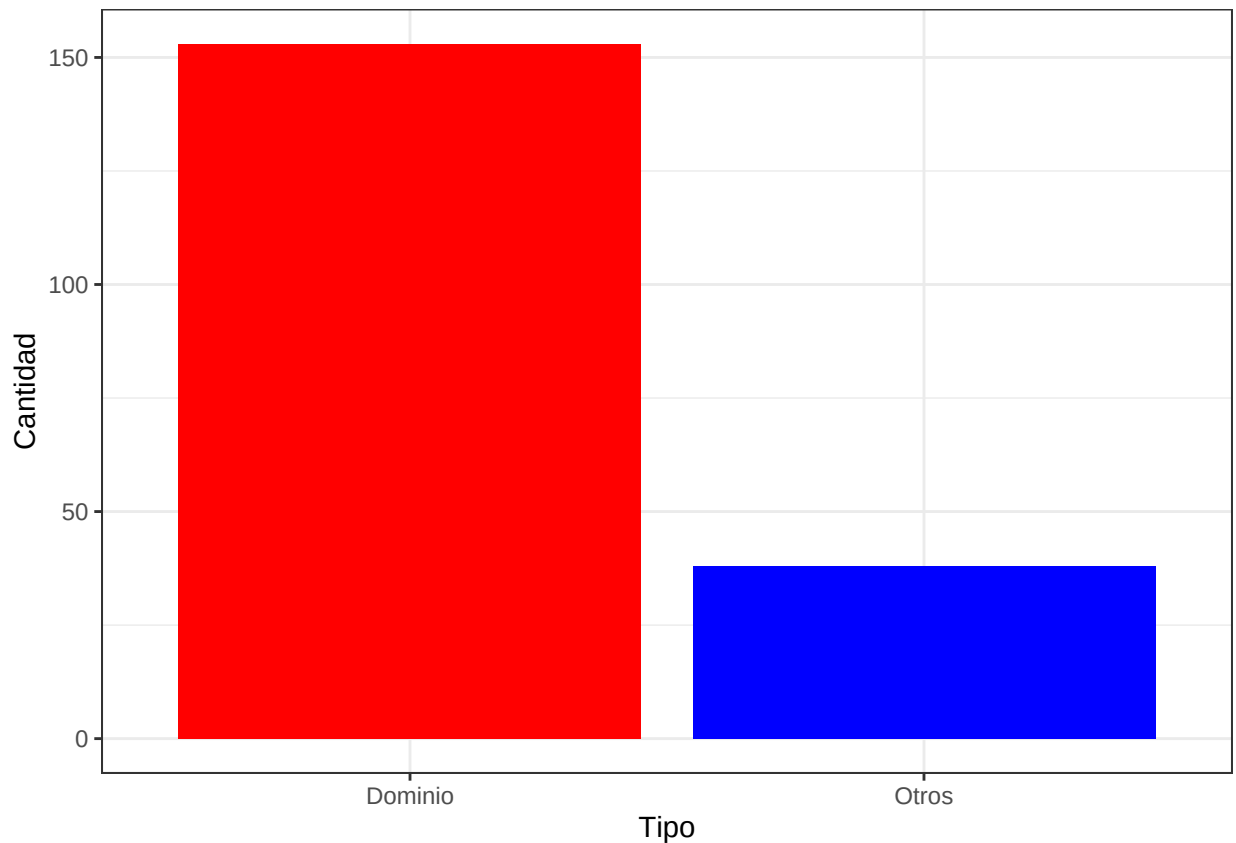
```
## [1] 191
```

```
dominio + otros
```

```
## [1] 191
```

Luego creamos un *barplot* que representará el pequeño *data frame* que acabamos de formar con estas dos variables y lo representamos gráficamente usando la librería de *ggplot2*.

```
dominio_o_no <- data.frame(  
  Tipo=c("Dominio", "Otros"),  
  Cantidad=c(dominio, otros)  
)  
  
barplot_dominio <- ggplot(dominio_o_no, aes(x=Tipo, y=Cantidad, fill=Tipo)) +  
  geom_col() +  
  scale_fill_manual(values = c("red", "blue")) +  
  theme_bw() +  
  theme(legend.position="none")  
  
barplot_dominio
```



3. Un gráfico de tarta (*pie chart*) indicando los porcentajes de Status de nuestro análisis. Por ejemplo, si hay 6 enlaces con status “200” y 4 enlaces con status “404”, la tarta mostrará un 60% con la etiqueta “200” y un 40% con la etiqueta “404”. Este gráfico lo uniremos a los anteriores. El objetivo final es obtener una imagen que recopile los gráficos generados.

Para este último ejercicio hacemos algo parecido al anterior, donde en este caso solo realizamos el conteo de qué URL tenían *status_code* 200 y cuáles tenían *status_code* 404.

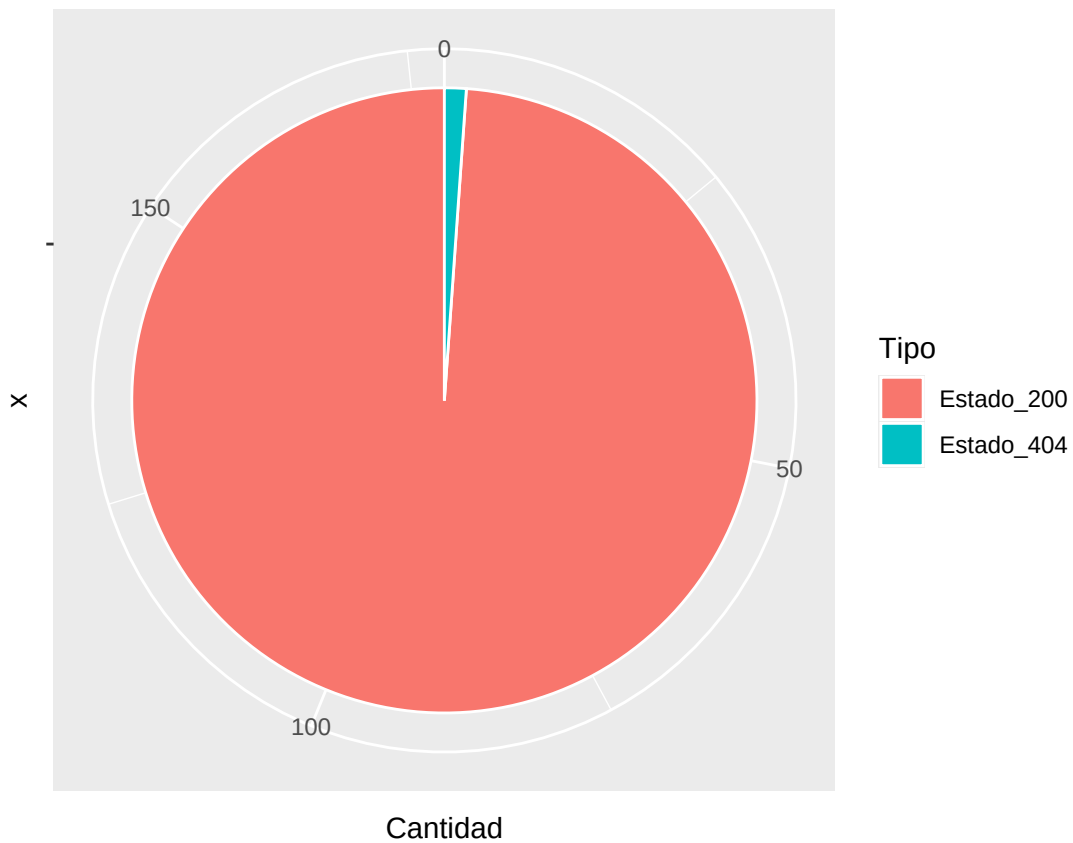
```
estado_200 <- count(datos[str_starts(datos$Estado, "200"), ])
estado_404 <- count(datos[str_starts(datos$Estado, "404"), ])
```

Y finalmente realizamos el mismo proceso de antes, pero ahora creamos un *Pie Chart*.

```
# Basic piechart
estados <- data.frame(
  Tipo=c("Estado_200", "Estado_404"),
  Cantidad=c(estado_200[1,1], estado_404[1,1])
)

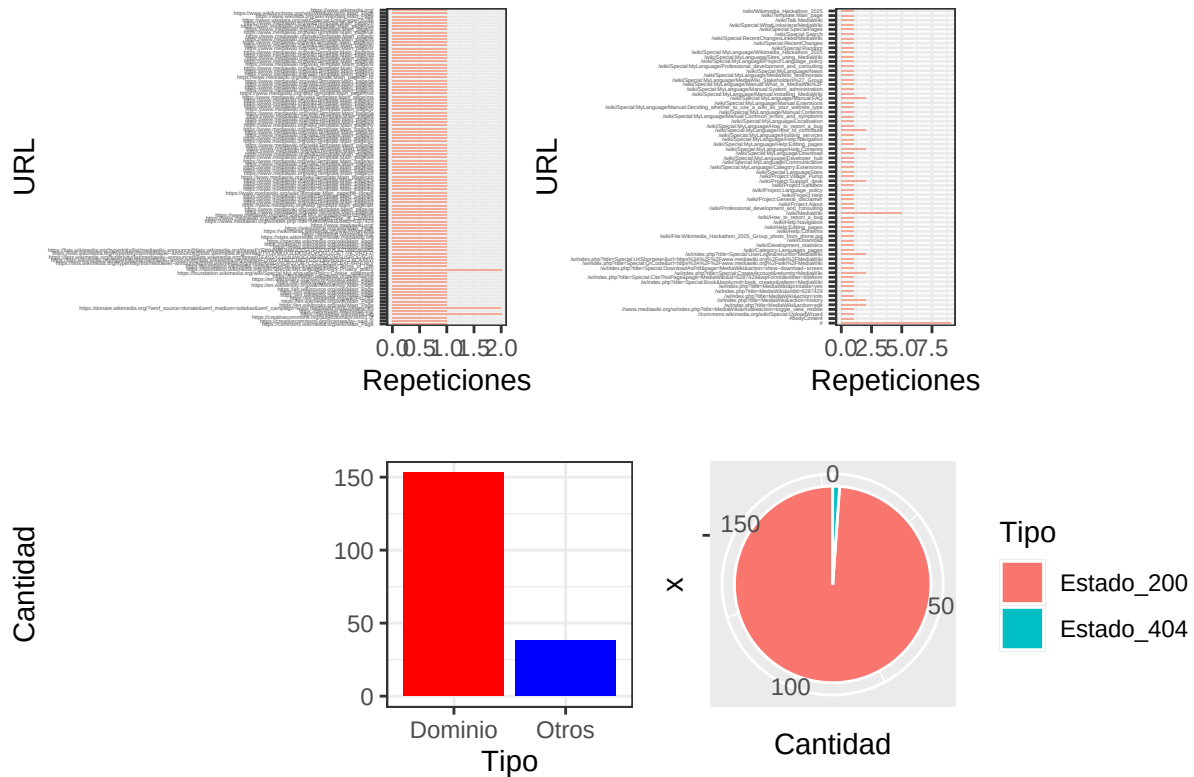
pie_chart <- ggplot(estados, aes(x="", y=Cantidad, fill=Tipo)) +
  geom_bar(stat="identity", width=1, color="white") +
  coord_polar("y", start=0)

pie_chart
```



Para terminar, aqui tenemos todas las graficas en una misma imagen:

```
grafico_final <- (barplot_absolutas + barplot_relativas) / (barplot_dominio + pie_chart)
grafico_final
```



Aquí está el código para poder descargar las imágenes como png:

```
# BARPLOT ABSOLUTAS
png("barplot_absolutas.png", width = 12000, height = 9000, res = 1000)
print(barplot_absolutas)
dev.off()
```

```
## pdf
## 2
```

```
# BARPLOT RELATIVAS
png("barplot_relativas.png", width = 12000, height = 9000, res = 1000)
print(barplot_relativas)
dev.off()
```

```
## pdf
## 2
```

```
# BARPLOT DOMINIO
png("barplot_dominio.png", width = 2400, height = 1800, res = 200)
print(barplot_dominio)
dev.off()
```

```
## pdf
## 2
```

```
# PIE CHART
png("pie_chart.png", width = 2400, height = 1800, res = 200)
print(pie_chart)
dev.off()
```

```
## pdf
## 2
```

```
# GRAFICO FINAL
png("grafico_final.png", width = 24000, height = 18000, res = 2000)
print(grafico_final)
dev.off()
```

```
## pdf
## 2
```